

Московский государственный технический
университет им. Н. Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”

Курс “Парадигмы и конструкции языков программирования”
Отчёт по лабораторной работе №4

Выполнил:
Студент группы ИУ5-31Б
Луценко Александра

Проверил:
Гапанюк Ю.Е

Adaptar_pattern.py

```
class LegacyPrinter:
    def print_document_legacy(self, text):
        return f"Печать (старый формат): {text}"

class ModernPrinter:
    def print_document(self, content):
        return f"Печать (современный формат): {content}"

class PrinterAdapter:
    def __init__(self, legacy_printer):
        self.legacy_printer = legacy_printer

    def print_document(self, content):
        return self.legacy_printer.print_document_legacy(content)
```

factory_pattern.py

```
from abc import ABC, abstractmethod
```

```
class Document(ABC):
    @abstractmethod
    def open(self):
        pass

    @abstractmethod
    def save(self):
        pass

class PDFDocument(Document):
    def open(self):
        return "Открыт PDF документ"

    def save(self):
        return "PDF документ сохранен"

class WordDocument(Document):
    def open(self):
        return "Открыт Word документ"

    def save(self):
        return "Word документ сохранен"

class DocumentFactory:
    @staticmethod
```

```

def create_document(doc_type):
    if doc_type == 'pdf':
        return PDFDocument()
    elif doc_type == 'word':
        return WordDocument()
    else:
        raise ValueError(f'Неизвестный тип документа: {doc_type}')

```

main.py

```

from factory_pattern import DocumentFactory
from adapter_pattern import LegacyPrinter, PrinterAdapter, ModernPrinter
from observer_pattern import NewsAgency, NewsSubscriber

```

```

def demonstrate_factory_pattern():
    print("=== ФАБРИЧНЫЙ МЕТОД ===")

    pdf_doc = DocumentFactory.create_document('pdf')
    word_doc = DocumentFactory.create_document('word')

    print(f'PDF документ: {pdf_doc.open()} -> {pdf_doc.save()}')
    print(f'Word документ: {word_doc.open()} -> {word_doc.save()}')

    try:
        DocumentFactory.create_document('excel')
    except ValueError as e:
        print(f'Ошибка: {e}')

```

```

def demonstrate_adapter_pattern():
    print("\n=== АДАПТЕР ===")

    legacy_printer = LegacyPrinter()
    modern_printer = ModernPrinter()
    adapter = PrinterAdapter(legacy_printer)

    print("Современный принтер:", modern_printer.print_document("Документ 1"))
    print("Адаптер для старого принтера:", adapter.print_document("Документ 2"))
    print("Старый принтер напрямую:",
    legacy_printer.print_document_legacy("Документ 3"))

```

```

def demonstrate_observer_pattern():
    print("\n=== НАБЛЮДАТЕЛЬ ===")

```

```

agency = NewsAgency()

subscribers = [
    NewsSubscriber("Иван Иванов"),
    NewsSubscriber("Мария Петрова"),
    NewsSubscriber("Алексей Сидоров")
]

for subscriber in subscribers[:2]:
    agency.attach(subscriber)
    print(f"{subscriber.name} подписался на новости")

print("\n--- Публикация первой новости ---")
agency.publish_news("Python 3.12 выпущен!")

agency.attach(subscribers[2])
print(f"\n{subscribers[2].name} подписался на новости")

print("\n--- Публикация второй новости ---")
agency.publish_news("Новый фреймворк для тестирования")

agency.detach(subscribers[1])
print(f"\n{subscribers[1].name} отписался от новостей")

print("\n--- Публикация третьей новости ---")
agency.publish_news("Итоги конференции PyCon")

print("\n--- Статистика полученных новостей ---")
for subscriber in subscribers:
    print(f"{subscriber.name}: {len(subscriber.received_news)} новостей")

if __name__ == "__main__":
    demonstrate_factory_pattern()
    demonstrate_adapter_pattern()
    demonstrate_observer_pattern()

```

observer_pattern.py

```

from abc import ABC, abstractmethod

```

```

class Observer(ABC):
    @abstractmethod
    def update(self, message):
        pass

```

```

class Subject(ABC):
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)

    def detach(self, observer):
        if observer in self._observers:
            self._observers.remove(observer)

    def notify(self, message):
        for observer in self._observers:
            observer.update(message)

class NewsAgency(Subject):
    def __init__(self):
        super().__init__()
        self._latest_news = None

    def publish_news(self, news):
        self._latest_news = news
        self.notify(f"Новая новость: {news}")

    def get_latest_news(self):
        return self._latest_news

class NewsSubscriber(Observer):
    def __init__(self, name):
        self.name = name
        self.received_news = []

    def update(self, message):
        self.received_news.append(message)
        return f"{self.name} получил: {message}"

```

test_adapter.py

```

import unittest
from adapter_pattern import LegacyPrinter, ModernPrinter, PrinterAdapter

class TestAdapterPattern(unittest.TestCase):

    def test_modern_printer(self):

```

```

printer = ModernPrinter()
result = printer.print_document("Тест")
self.assertEqual(result, "Печать (современный формат): Тест")

def test_legacy_printer(self):
    printer = LegacyPrinter()
    result = printer.print_document_legacy("Тест")
    self.assertEqual(result, "Печать (старый формат): Тест")

def test_adapter(self):
    legacy_printer = LegacyPrinter()
    adapter = PrinterAdapter(legacy_printer)
    result = adapter.print_document("Тест")
    self.assertEqual(result, "Печать (старый формат): Тест")

if __name__ == '__main__':
    unittest.main()

```

test_bdd_style.py
import unittest

```

class BDDStyleTests(unittest.TestCase):

    def test_document_factory_creation_flow(self):
        from factory_pattern import DocumentFactory

        pdf_document = DocumentFactory.create_document('pdf')

        from factory_pattern import PDFDocument
        self.assertIsInstance(pdf_document, PDFDocument)
        self.assertEqual(pdf_document.open(), "Открыт PDF документ")
        self.assertEqual(pdf_document.save(), "PDF документ сохранен")

        word_document = DocumentFactory.create_document('word')

        from factory_pattern import WordDocument
        self.assertIsInstance(word_document, WordDocument)
        self.assertEqual(word_document.open(), "Открыт Word документ")
        self.assertEqual(word_document.save(), "Word документ сохранен")

    def test_adapter_pattern_flow(self):
        from adapter_pattern import LegacyPrinter, PrinterAdapter

        legacy_printer = LegacyPrinter()

```

```

adapter = PrinterAdapter(legacy_printer)

result = adapter.print_document("Тестовый документ")

expected = "Печать (старый формат): Тестовый документ"
self.assertEqual(result, expected)

def test_observer_pattern_flow(self):
    from observer_pattern import NewsAgency, NewsSubscriber

    agency = NewsAgency()
    subscriber1 = NewsSubscriber("Иван")
    subscriber2 = NewsSubscriber("Мария")

    agency.attach(subscriber1)
    agency.attach(subscriber2)
    agency.publish_news("Важная новость!")

    self.assertEqual(len(subscriber1.received_news), 1)
    self.assertEqual(len(subscriber2.received_news), 1)

    agency.detach(subscriber1)
    agency.publish_news("Вторая новость")

    self.assertEqual(len(subscriber1.received_news), 1)
    self.assertEqual(len(subscriber2.received_news), 2)

if __name__ == '__main__':
    unittest.main()

```

test_error_cases.py

```

import unittest
import sys

def run_all_tests():
    loader = unittest.TestLoader()
    start_dir = '.'
    suite = loader.discover(start_dir, pattern='test_*.py')

    runner = unittest.TextTestRunner(verbosity=2)
    result = runner.run(suite)

    return result.wasSuccessful()

```

```
if __name__ == '__main__':
    success = run_all_tests()
    sys.exit(0 if success else 1)
```

test_factory_tdd.py

```
import unittest
from factory_pattern import DocumentFactory, PDFDocument, WordDocument
```

```
class TestFactoryPattern(unittest.TestCase):
```

```
    def test_pdf_document_creation(self):
        document = DocumentFactory.create_document('pdf')
        self.assertIsInstance(document, PDFDocument)
```

```
    def test_word_document_creation(self):
        document = DocumentFactory.create_document('word')
        self.assertIsInstance(document, WordDocument)
```

```
    def test_invalid_document_type(self):
        with self.assertRaises(ValueError):
            DocumentFactory.create_document('excel')
```

```
    def test_pdf_document_methods(self):
        document = PDFDocument()
        self.assertEqual(document.open(), "Открыт PDF документ")
        self.assertEqual(document.save(), "PDF документ сохранен")
```

```
    def test_word_document_methods(self):
        document = WordDocument()
        self.assertEqual(document.open(), "Открыт Word документ")
        self.assertEqual(document.save(), "Word документ сохранен")
```

```
if __name__ == '__main__':
    unittest.main()
```

test_mock_objects.py

```
import unittest
from unittest.mock import Mock, patch, MagicMock
```

```
class TestWithMockObjects(unittest.TestCase):
```

```
    def test_observer_with_mocks(self):
```



```

mock_observer1 = Mock()
mock_observer2 = Mock()

from observer_pattern import NewsAgency
agency = NewsAgency()

agency.attach(mock_observer1)
agency.attach(mock_observer2)

test_news = "Тестовая новость"
agency.publish_news(test_news)

expected_message = f"Новая новость: {test_news}"
mock_observer1.update.assert_called_once_with(expected_message)
mock_observer2.update.assert_called_once_with(expected_message)

def test_adapter_with_mock(self):
    mock_legacy_printer = Mock()
    mock_legacy_printer.print_document_legacy.return_value = "Mocked print
result"

    from adapter_pattern import PrinterAdapter
    adapter = PrinterAdapter(mock_legacy_printer)

    result = adapter.print_document("Test content")

    mock_legacy_printer.print_document_legacy.assert_called_once_with("Test
content")
    self.assertEqual(result, "Mocked print result")

def test_factory_with_patch(self):
    with patch('factory_pattern.PDFDocument') as MockPDFDocument:
        mock_instance = Mock()
        mock_instance.open.return_value = "Mocked open"
        mock_instance.save.return_value = "Mocked save"
        MockPDFDocument.return_value = mock_instance

    from factory_pattern import DocumentFactory
    document = DocumentFactory.create_document('pdf')

    MockPDFDocument.assert_called_once()

    self.assertEqual(document.open(), "Mocked open")
    self.assertEqual(document.save(), "Mocked save")

```

```

def test_news_agency_integration_with_mock(self):
    from observer_pattern import NewsAgency, NewsSubscriber

    agency = NewsAgency()

    real_subscriber = NewsSubscriber("Реальный подписчик")
    mock_subscriber = Mock()

    agency.attach(real_subscriber)
    agency.attach(mock_subscriber)

    agency.publish_news("Интеграционная новость")

    self.assertEqual(len(real_subscriber.received_news), 1)

    mock_subscriber.update.assert_called_once()

if __name__ == '__main__':
    unittest.main()

```

test_observer.py

```

import unittest
from observer_pattern import NewsAgency, NewsSubscriber

class TestObserverPattern(unittest.TestCase):

    def test_attach_detach(self):
        agency = NewsAgency()
        subscriber = NewsSubscriber("Тест")

        agency.attach(subscriber)
        self.assertIn(subscriber, agency._observers)

        agency.detach(subscriber)
        self.assertNotIn(subscriber, agency._observers)

    def test_notification(self):
        agency = NewsAgency()
        subscriber1 = NewsSubscriber("Подписчик 1")
        subscriber2 = NewsSubscriber("Подписчик 2")

        agency.attach(subscriber1)
        agency.attach(subscriber2)

```

```

agency.publish_news("Тестовая новость")

self.assertEqual(len(subscriber1.received_news), 1)
self.assertEqual(len(subscriber2.received_news), 1)
self.assertIn("Тестовая новость", subscriber1.received_news[0])

def test_subscriber_update(self):
    subscriber = NewsSubscriber("Тест")
    result = subscriber.update("Сообщение")
    self.assertEqual(result, "Тест получил: Сообщение")

if __name__ == '__main__':
    unittest.main()

```

```

=== ФАБРИЧНЫЙ МЕТОД ===
PDF документ: Открыт PDF документ -> PDF документ сохранен
Word документ: Открыт Word документ -> Word документ сохранен
Ошибка: Известный тип документа: excel

=== АДАПТЕР ===
Современный принтер: Печать (современный формат): Документ 1
Адаптер для старого принтера: Печать (старый формат): Документ 2
Старый принтер напрямую: Печать (старый формат): Документ 3

=== НАБЛЮДАТЕЛЬ ===
Иван Иванов подписался на новости
Мария Петрова подписался на новости

--- Публикация первой новости ---

Алексей Сидоров подписался на новости

--- Публикация второй новости ---

Мария Петрова отписался от новостей

--- Публикация третьей новости ---

--- Статистика полученных новостей ---
Иван Иванов: 3 новостей
Мария Петрова: 2 новостей
Алексей Сидоров: 2 новостей

```